

Unit IV

Deadlock, Device Management & File System

Introduction

In a multiprogramming environment, several processes may compete for a finite number of resources. A process requests resources; if the resources are not available at that time, the process enters a waiting state. Sometimes, a waiting process is never again able to change state, because the resources it has requested are held by other waiting processes. This situation is called **deadlocks**.

In this chapter, we describe methods that an operating system can use to prevent or deal with deadlocks. Although some applications can identify programs that may deadlock, operating systems typically do not provide deadlock-prevention facilities, and it remains the responsibility of application programmers to ensure that they design deadlock-free programs. Deadlock problems can only become more common, given current trends, including larger numbers of processes, multithreaded programs, many more resources within a system, and an importance on long-lived file and database servers rather than batch systems.

System Model

A process must request a resource before using it and must release resource after using it. A process may request as many resources as it requires carrying out its designated task. Obviously the number of resources requested may not exceed the total number of resources available in the system. In other words, a process cannot request three printers if the system has only two.

Under the normal mode of operation, a process may utilize a resource in only the following sequence:

Request: The process requests the resource. If the request cannot be granted immediately (for example, if the resource is being used by another process), then the requesting process must wait until it can acquire the resource.

Use: The process can operate on the resource (for example, if the resource is a printer, the process can print on the printer).

Release: The process releases the resource.

As we know request and release of resources are system calls, as. Examples are the `Request()` & `Release()` for device, `Open()` & `Close()` for file, and `Allocate()` & `Free()` are the memory system calls.

In a computer system, several processes run simultaneously and compete for system resources such as memory, files, printers, or CPU time. A deadlock happens when a group of processes gets into a situation where every process is waiting for another process to release a resource. Since none of them release the resources they already hold, all the processes remain waiting forever.

In simple terms, deadlock is a state in which processes block each other permanently because each process is waiting for a resource held by another process.

Example of Deadlock

Consider a system with two processes **P1** and **P2**, and two resources **R1** and **R2**.

- Process **P1** holds resource **R1** and is waiting for **R2**.
- Process **P2** holds resource **R2** and is waiting for **R1**.

In this situation:

- P1 cannot proceed until it gets R2
- P2 cannot proceed until it gets R1

Since both processes are waiting for each other, neither of them can continue, and the system enters a deadlock state.

Deadlock Characterization

In a deadlock, processes never finish executing and system resources tied up, preventing other jobs from starting. Before we discuss the various methods for dealing with the deadlock problem, we focus closely at features that characterize deadlocks.

Necessary Conditions for deadlocks

Lock situation can arise if the following four conditions hold simultaneously in a system.

Mutual Exclusion: At least one resource must be held in non-sharable mode; that is only one process at a time can use the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.

Hold and Wait: A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.

No preemption: Resources cannot be preempted; that is resource can be released only voluntarily by the process holding it, after that process completed its task.

Circular wait: A set $\{P_0, P_1, P_2, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1 , P_1 is waiting for a resource held by P_2 , ..., P_{n-1} is waiting for a resource held by P_n and P_n is waiting for process P_0 .

Dead lock Preventions

Deadlock Prevention is a technique used in Operating Systems to ensure that a deadlock situation never occurs in the system. The basic idea of deadlock prevention is to design the system in such a way that at least one of the necessary conditions for deadlock cannot occur.

A deadlock occurs only when four conditions exist simultaneously: mutual exclusion, hold and wait, no preemption, and circular wait. Deadlock prevention works by eliminating one or more of these conditions.

1. Prevention of Mutual Exclusion

The mutual exclusion condition states that only one process can use a resource at a time. To prevent deadlock, the system tries to make resources sharable instead of exclusive.

For example, read-only files can be shared by multiple processes at the same time without conflict. If resources are sharable, the mutual exclusion condition does not exist and deadlock cannot occur.

However, this method cannot be applied to all resources. Some resources like printers or devices must remain exclusive, so this method has limited applicability.

2. Prevention of Hold and Wait

The hold and wait condition occurs when a process holds one resource while waiting for another resource. To prevent this, the operating system may require that a process request all the resources it needs at one time before execution begins. If all resources are available, the process is allocated them and starts execution. If not, it must wait until all required resources become available.

Another method is that a process must release all currently held resources before requesting new ones. This ensures that a process never holds resources while waiting for others.

The disadvantage of this approach is that it reduces resource utilization and system efficiency, because resources may remain unused while processes wait.

3. Prevention of No Preemption

The no preemption condition means that resources cannot be forcibly taken from a process. To prevent deadlock, the operating system may allow preemption of resources. If a process holding certain resources requests another resource that is not available, the system may force the process to release the resources it currently holds. These resources are then allocated to other processes.

The process can continue execution only when it successfully reacquires all the required resources.

4. Prevention of Circular Wait

Circular wait occurs when a chain of processes exists where each process is waiting for a resource held by another process in the chain.

To prevent this, the system assigns a unique ordering or numbering to all resources. Processes must request resources only in increasing order of their numbers.

For example, if resource numbers are R1, R2, and R3, a process that holds R1 can request R2 or R3 but cannot request a resource with a lower number than the one it already holds. This prevents the formation of a circular chain.

Deadlock prevention ensures that deadlocks never occur by restricting the way processes request and use resources. Although it improves system safety, it may also reduce system performance and resource utilization because processes must follow strict rules when requesting resources.

Deadlock Avoidance

Deadlock Avoidance is a technique used in Operating Systems to ensure that the system never enters a deadlock state. Unlike deadlock prevention, which restricts the way resources are requested, deadlock avoidance carefully examines each resource request and decides whether granting the request will lead the system into a safe or unsafe state.

The main idea behind deadlock avoidance is that the operating system predicts the possibility of deadlock before allocating resources. If granting a resource request could potentially lead to a deadlock situation, the operating system delays the request until it is safe to allocate the resource.

Safe State and Unsafe State

Deadlock avoidance is based on the concept of safe state and unsafe state.

A safe state is a condition in which the system can allocate resources to each process in some order and still avoid a deadlock. In other words, there exists at least one sequence of process execution (called a "safe sequence") in which every process can obtain the resources it needs and complete its execution.

An unsafe state is a state where the system does not have a guaranteed safe sequence. This does not always mean that a deadlock has already occurred, but it means that the system may enter a deadlock in the future if resource allocation continues without control.

Therefore, in deadlock avoidance, the operating system allocates resources only if the system remains in a safe state after the allocation.

Banker's Algorithm

One of the most well-known algorithms used for deadlock avoidance is the Banker's Algorithm. The name comes from the idea of a banker who manages money and ensures that loans are granted only if the bank can still satisfy all its customers.

In this algorithm, the operating system requires each process to declare the maximum number of resources it may need during its execution. Using this information, the system keeps track of the following:

- The maximum demand of each process
- The currently allocated resources
- The remaining resource needs
- The available resources in the system

When a process requests a resource, the system temporarily allocates the resource and checks whether the system remains in a safe state. If the system remains safe, the allocation is confirmed. If it leads to an unsafe state, the request is denied and the process must wait.

Advantages of Deadlock Avoidance

Deadlock avoidance ensures that the system never enters a deadlock situation and allows resources to be allocated more flexibly compared to strict prevention techniques.

Disadvantages of Deadlock Avoidance

However, deadlock avoidance requires advance knowledge of the maximum resource requirements of each process, which may not always be possible. It also requires continuous checking of system states, which can increase system overhead.

Deadlock avoidance is an effective method in operating systems for managing resources safely. By allocating resources only when the system remains in a safe state, the operating system can prevent deadlocks while still allowing efficient resource utilization.

Deadlock detection & Recovery

In Operating Systems, a deadlock may still occur if the system does not use prevention or avoidance techniques. In such situations, the operating system allows the deadlock to occur but provides mechanisms to detect the deadlock and recover from it. This approach is called deadlock detection and recovery.

Deadlock detection and recovery involve two main steps. First, the operating system must identify whether a deadlock has occurred. After detecting the deadlock, the system must take appropriate action to break the deadlock and restore normal system operation.

Deadlock Detection

Deadlock detection is the method used by the operating system to check the system state and determine whether processes are deadlocked.

Wait-for Graph Method

When each resource type has only one instance, the operating system uses a Wait-for Graph to detect deadlocks.

In a wait-for graph:

- Nodes represent processes.
- Edges represent waiting relationships between processes.

For example, if process P1 is waiting for a resource that is currently held by process P2, a directed edge is drawn from P1 to P2.

If the graph contains a **cycle**, it means that the processes involved in the cycle are waiting for each other. In this case, a deadlock exists in the system.

Detection with Multiple Resource Instances

When resources have multiple instances, the operating system uses a deadlock detection algorithm that works similarly to the Banker's safety algorithm.

The algorithm examines:

- The currently allocated resources,
- The available resources, and
- The outstanding resource requests of each process.

By analyzing this information, the system determines whether any processes cannot proceed and are permanently waiting for resources held by others.

Deadlock Recovery

Once the operating system detects a deadlock, it must recover from the deadlock by breaking the circular waiting condition. There are two main methods for recovery.

1. Process Termination

In this method, the operating system terminates one or more processes involved in the deadlock so that their resources are released.

Two approaches can be used:

- Abort all deadlocked processes, which immediately free all resources but causes loss of work.
- Abort one process at a time until the deadlock cycle is eliminated.

The system chooses which process to terminate based on factors such as process priority, amount of work completed, resources used, and the cost of restarting the process.

2. Resource Preemption

Resource preemption means taking resources away from some processes and giving them to other processes until the deadlock is resolved.

This method involves:

- Selecting a victim process from which resources will be taken.
- Rolling back the process to a previous safe state.
- Restarting the process later when the required resources become available.

Although this method can resolve deadlocks, it may lead to starvation, where the same process repeatedly loses its resources.

Deadlock detection and recovery allow the operating system to handle deadlocks after they occur instead of preventing them beforehand. The system first detects deadlocks by analyzing process-resource relationships and then recovers by terminating processes or preempting resources. Although effective, this approach may lead to performance overhead and loss of computation, so it must be applied carefully.

Techniques for Device Management channels & Control Unit

In Operating System, Device Management is responsible for controlling and coordinating input and output (I/O) devices such as keyboards, printers, disks, and monitors. The operating system must ensure that these devices work efficiently and that multiple processes can use them without conflict. Two important concepts used in device management are Channels and Control Units.

Channels

A **channel** is a specialized hardware component that acts as an I/O processor. It manages communication between the CPU and I/O devices. Instead of the CPU directly controlling every device operation, the CPU assigns the task to the channel, and the channel performs the I/O operation independently.

In early computer systems, the CPU had to manage all input and output operations, which slowed down the overall performance. To solve this problem, channels were introduced to offload I/O tasks from the CPU.

A channel works as an intermediary. When a program needs to perform an input or output operation, the CPU sends a command to the channel. The channel then handles the transfer of data between memory and the device while the CPU continues executing other instructions. This improves system efficiency and allows parallel execution of computation and I/O operations.

Types of Channels

Selector Channel: A selector channel manages one device at a time but transfers data at a high speed. It selects a single I/O device and remains dedicated to that

device until the entire data transfer is completed. This type of channel is usually used for high-speed devices such as disk drives.

2. Multiplexer Channel: A multiplexer channel can handle multiple devices simultaneously by interleaving their data transfers. Instead of dedicating itself to one device, it divides its time among several devices. This is useful for slow-speed devices like keyboards, printers, and terminals.

Thus, channels reduce CPU workload, improve system throughput, and allow efficient handling of I/O operations.

2. Control Unit (Device Controller/Device Management)

A control unit, often called a device controller, is a hardware component that manages a specific I/O device. Every device usually has its own controller that acts as a bridge between the device and the computer system.

The controller receives commands from the operating system through the channel or CPU. It interprets these commands and controls the actual device operations such as reading data, writing data, or transferring data to memory.

A control unit generally contains:

- **Control logic** - manages device operations
- **Registers** - store commands, status information, and data
- **Buffer memory** - temporarily stores data during transfer

For example, a disk controller manages the operations of a hard disk. When the OS requests data from the disk, the disk controller moves the read/write head, retrieves the data, and transfers it to main memory.

The control unit also reports the device status back to the operating system. If an error occurs during an operation, the controller informs the OS so that corrective action can be taken.

Relationship between Channel and Control Unit

Channels and control units work together to perform I/O operations efficiently.

1. The CPU sends an I/O request to the channel.
2. The channel communicates with the control unit of the device.
3. The control unit manages the actual device and performs the data transfer.
4. Once the operation is complete, the status is returned to the CPU.

This layered structure ensures that the CPU does not need to directly manage the hardware details of each device.

Channels and control units are important techniques used in device management. A channel acts as an I/O processor that handles data transfer between memory and devices, reducing CPU workload. A control unit (device controller) manages the actual functioning of a specific hardware device and executes commands sent by the operating system. Together they improve efficiency, speed, and reliability in computer systems.

File System & General Model of File System

A File System is an important part of an Operating System (OS) that is responsible for organizing, storing, managing, and retrieving data on storage devices such as hard disks, SSDs, and USB drives. It provides a logical way to store information so that users and programs can easily access and manage files.

A **file** is a collection of related information stored on secondary storage. This information may include text, images, programs, videos, or any other data. The file system is the method used by the operating system to store and organize these files in a structured manner.

Without a file system, all data would be stored randomly on the disk, making it very difficult to find or retrieve information. The file system creates a structured environment where files are stored with names, locations, and attributes.

Objectives of File System: The main objectives of a file system are:

1. To store data permanently on secondary storage.
2. To organize files efficiently so that they can be easily accessed.
3. To provide a logical view of data to users.
4. To allow sharing of files between different users or programs.
5. To protect data from unauthorized access.

General Model of a File System

The General Model of a File System describes how an Operating System (OS) organizes and manages files stored on secondary storage devices such as disks. It explains the different layers and components involved in storing, accessing, and controlling files. The model shows how users interact with files and how the operating system internally manages the storage and retrieval of data.

In a computer system, users do not directly access the physical disk. Instead, the operating system provides a structured model that hides the hardware complexity and offers an easy way to manage files. The general model of a file system can be understood as a **layered structure**, where each layer performs a specific function.

Layers in the General Model of a File System

1. User Level

At the top level are the users and application programs. Users interact with the file system through commands such as creating, opening, reading, writing, and deleting files. Application programs also request file operations through system calls provided by the operating system.

For example, when a user saves a document or opens a file, the request is sent to the operating system.

2. Logical File System

The logical file system manages the logical structure of files and directories. It is responsible for maintaining information about files such as:

- File name
- File size
- File type
- File permissions
- File location

This layer also manages the directory structure, which organizes files in folders or directories. The logical file system ensures that users can easily locate and access files.

3. File Organization Module

The file organization module determines how files are physically stored on the disk. It translates the logical file addresses into physical disk addresses.

This module also manages file allocation methods, such as:

- Contiguous allocation
- Linked allocation
- Indexed allocation

Its main function is to allocate disk blocks for files and manage free space on the disk.

4. Basic File System

The basic file system handles the actual transfer of data between memory and the storage device. It issues commands such as read block or write block to the device drivers.

This layer works closely with the I/O control system of the operating system.

5. I/O Control

The I/O control layer consists of device drivers and interrupts handlers that control the hardware devices. It converts high-level file operations into hardware-specific instructions.

Device drivers communicate directly with the disk controller to perform operations like reading and writing data.

6. Hardware (Disk Storage)

At the lowest level is the physical storage device, such as a hard disk or solid-state drive. Data is stored in blocks or sectors on the disk surface. The operating system manages how these blocks are used to store files.

Important Question (Assignment IV)

Q.1: What is deadlock? Explain deadlock characterization.

Q.2: Explain deadlock preventions in detail.

Q.3: Explain deadlock avoidance.

Q.4: Explain Bankers algorithm.

Q.5: Explain deadlock detection in detail.

Q.6: Explain deadlock recovery in detail.

Q.7: Explain Channels and Control Unit.

Q.8: Explain general model of file system.